FRUIT CLASSIFICATION WITH TRANSFER LEARNING

December 5, 2025

ABSTRACT

This project investigates the problem of multi-class fruit classification using convolutional neural networks and transfer learning. The goal is to demonstrate an iterative, experimental workflow for improving model performance. Using a dataset which contains approximately 30,000 images across 30 fruit categories, we begin with a simple baseline CNN trained from scratch. The baseline exhibits overfitting and struggles with visually similar classes, which leads us to data augmentation. Building on these findings, we employ a ResNet50 backbone with transfer learning and further refine the model through a custom soft cost-sensitive loss function designed to penalize specific class confusions, as well as partial unfreezing of higher-level pretrained layers with discriminative learning rates. At the end, we compare the performance of the baseline and final models.

Keywords Deep Learning · Classification

1 Introduction

The goal of this project is to classify images of fruits using convolutional neural networks. There are several fields which need automated fruit classification. Firstly, when it comes to cooking, smart fridges could use fruit classification to identify which fruits were added or removed, create meal plans based on the types of fruits that we currently have and notify the user when a stock is low or when a certain type of fruit is about to go bad.

Self-checkout systems in shops or smart scales could automatically identify the fruit a customer places on the weighing station without the customer needing to search for them manually, which would reduce the checkout time.

In agriculture, fruit classification has a big potential as well. Farmers can use classification systems to monitor fruit growth, evaluate ripeness, and detect defects or diseases early. This helps them make informed decisions about harvest times. Automated classification can also support sorting and grading processes after harvest, ensuring that fruits are grouped by size, color, and quality with greater accuracy than when sorted manually.

There are some challenges that we need to face when dealing with fruit classification. Fruits are often similar in color or shape and since we trained our model on a relatively small dataset, there is a risk of overfitting. Moreover, fruit can be photographed with different background and from different angles.

To solve fruit classification problem, this report uses a Kaggle dataset containing approximately 30,000 images across 30 different fruit categories. The dataset is described in more detail in Section 3.1.

The main objective of this project is to determine whether we can systematically improve classification performance through motivated, theory-driven experiments, rather than through arbitrary hyperparameter tuning. Each stage of the model development is therefore guided by observations from the previous step: if the baseline overfits, we introduce data augmentation; if the model appears underpowered, we increase its capacity; if certain classes remain difficult to separate, we analyze confusion matrices and consider targeted interventions.

We begin with a simple baseline CNN trained from scratch to establish reference performance and reveal initial weaknesses such as overfitting. Guided by these findings, we introduce data augmentation, explore transfer learning using a pretrained ResNet50 backbone, analyze confusion patterns between visually similar classes, and design targeted interventions such as a custom confusion-penalty loss and partial layer unfreezing.

2 Related work

Fruit classification using deep learning has already been explored in several research studies. An example of such study is Fruits Classification and Detection Application Using Deep Learning by Mimma et al.Mimma et al. [2022] The authors evaluate YOLOv3 approach and VGGG16 and ResNet50 models for the task of fruit detection and classification and highlight the challenges caused by varying lighting conditions, irregular backgrounds, and similarities between fruit types.

Their experiments show that transfer learning with ResNet50, pretrained on ImageNet, achieves the highest classification accuracy among the tested models. This supports the idea that using a deep, pretrained architecture provides strong generalization even when the training dataset is relatively small. While our project begins with a simpler CNN baseline, the findings of Mimma et al. help us explore how the small size of the dataset influences learning.

Another important theme in the literature is the use of data augmentation to avoid overfitting. Many fruit-classification studies rely on augmentation (rotating, resizing, flipping) to artificially increase dataset diversity and improve generalization. The effectiveness of augmentation reported in previous work aligns with our observations in this project.

Authors of a second study, Amin et al. Amin et al. [2023], used transfer learning with the AlexNet architecture to classify fruit freshness across multiple publicly available datasets. Their method combines fine-tuning of a pretrained CNN with extensive preprocessing and data augmentation, achieving classification accuracies of over 98%.

Although their task focuses on freshness rather than fruit type, the underlying challenges are similar: fruits exhibit variation in color, texture, ripeness, and lighting, and the datasets contain limited examples per class. The strong performance obtained through transfer learning in their work reinforces the value of using pretrained models for fruit-related visual tasks and further motivates our decision to compare shallow CNNs with deeper architectures such as ResNet50 in later stages of this project.

Together, these studies demonstrate that fruit classification tasks benefit significantly from transfer learning and carefully designed augmentation strategies. They also highlight common difficulties, such as visually similar classes and inconsistent backgrounds, which are present in our dataset as well. The insights from these papers guide our experimental design: we begin with a simple baseline to expose the limitations of training from scratch, then incorporate augmentation and deeper architectures to evaluate whether the trends observed in prior research also apply to our dataset.

3 Methods

3.1 Dataset

The dataset used in this project is the Plants-Type Fruit Dataset Sulistya [2023], downloaded from Kaggle. It approximately 30,000 RGB images of 30 different fruit and vegetable categories. The images are organized into three subsets: 23,972 training images, 3,030 validation images, and 2,998 test images. Each subset includes all 30 classes, and the distribution across classes is nearly uniform. Figure 1 shows the class distribution for all three subsets, confirming that the dataset is well balanced.

A large portion of the dataset consists of augmented versions of existing images, identifiable by their filenames. These include transformations such as rotations or flips. For our initial baseline experiments, we excluded these pre-augmented images.

All images are JPEG files with varying sizes. To create a consistent input format, each image was resized to 224×224 pixels. The images contain only one fruit type per image, but backgrounds vary a lot: some images were taken on plain surfaces, while others include natural or cluttered backgrounds. Additionally, the photographs were taken from different angles and under different lighting conditions, contributing to the visual diversity of the dataset.

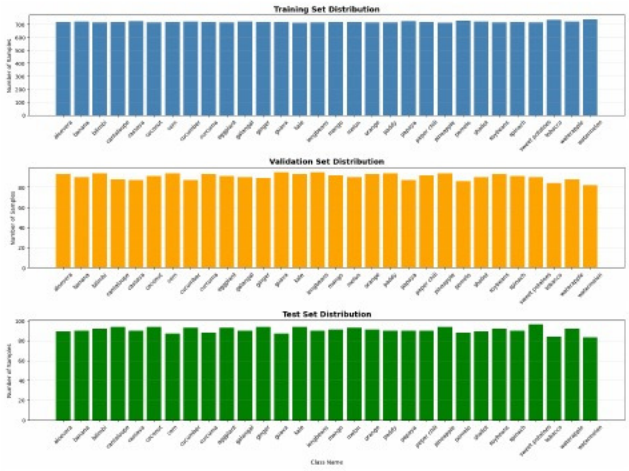

Figure 1: Training without augmented data

3.2 Implementation and Software Architecture

The project is implemented in Python using the PyTorch framework. The software architecture is designed around a central `Config` class that encapsulates all experimental parameters, ensuring strict separation between configuration and execution logic.

The execution flow begins with the `run_all` orchestrator, which iterates through a defined grid of hyperparameters. For each experiment, the code procedurally constructs the dataset environment. The `create_filtered_subsets` routine scans the raw data and builds a temporary directory structure containing only the target classes and specified image types (filtering out augmented files when disabled). This ensures the data loaders receive a clean input stream without requiring complex runtime filtering.

The model initialization logic dynamically adapts the architecture based on the configuration. The `build_model` function loads the ResNet50 backbone and programmatically freezes specific parameter groups while attaching a new linear classification head. During training, the optimizer is initialized with differential learning rates—applying a lower rate to the unfrozen backbone layers and a higher rate to the new head—to preserve pretrained features while learning task-specific boundaries. Finally, an evaluation module automatically compiles performance metrics into JSON history files and generates confusion matrices, creating a standardized set of artifacts for every experimental run.

3.3 Setup and Reproducibility

To ensure reproducibility and ease of setup, the project repository includes an automated environment configuration. To run the script, users must clone the repository and configure the local environment.

The system handles dataset management automatically using the Kaggle API. To facilitate this, a valid `kaggle.json` API token must be placed in the project root directory. Upon initiating the training pipeline—via either the "Solo Run" (for single configurations) or "Run All" (for automated hyperparameter sweeping) modes—the script checks for the presence of the data. If the dataset is missing, the system uses the credentials in `kaggle.json` to automatically download and extract the dataset from Kaggle, ensuring the correct directory structure is maintained.

```
# Clone repository and navigate to directory
git clone https://github.com/                                .git
cd Fruit-Classification

# Place kaggle.json in this directory
# ...

# Create and activate virtual environment
python -m venv .venv
# Linux/Mac: source .venv/bin/activate
# Windows:   .venv\Scripts\activate
```

```
# Install dependencies
pip install -r requirements.txt
```

Once the environment is active, open `Fruit_Classification_Project.ipynb` to access the training logic.

3.4 Adding data augmentation

The dataset used in this project contains both original and augmented images of fruits and vegetables. Augmented images are pre-generated variations of the original images, created using transformations such as rotation, flipping, colour adjustments, and cropping. These augmented images are identified by the "aug_" prefix in their filenames.

To evaluate the impact of data augmentation on model performance, we implemented functionality to toggle between training with original images only versus training with augmented images. This was achieved through a configurable parameter in the training pipeline IMG_AUGMENT. When set to True, the training pipeline selects images with the aug_ prefix; when set to False, only original images (without the prefix) are used.

3.4.1 Training Without Augmented Data

When training with original images only, the model achieved approximately 85% validation accuracy after 5 epochs 2. The training curves showed signs of potential overfitting, with a noticeable gap between training and validation accuracy. The confusion matrix revealed several problematic class pairs, particularly between visually similar fruits.

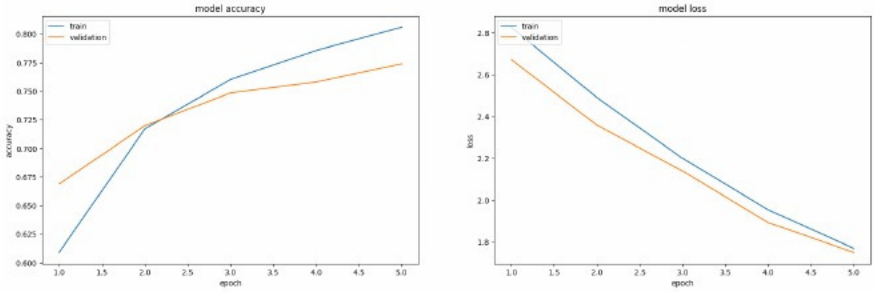

Figure 2: Training without augmented data

3.4.2 Training With Augmented Data

Training with augmented images yielded improved results, achieving approximately 87% validation accuracy under the same training conditions 3. The training curves demonstrated better generalisation, with the gap between training and validation metrics being smaller compared to the non-augmented scenario.

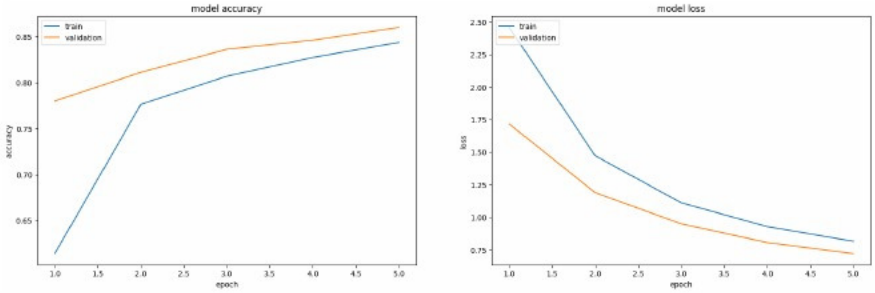

Figure 3: Training with augmented data

The improvement observed when using augmented data can be attributed to several factors. Data augmentation effectively increases the diversity of training samples, exposing the model to a wider variety of visual representations for each class. This helps the model learn more robust features that generalise better to unseen data. Based on these findings, all subsequent experiments in the model improvement process were conducted using the augmented dataset as the baseline.

3.5 Analyzing confusion matrix

After establishing the augmented dataset as the baseline, we generated confusion matrices to identify specific classification errors and understand where the model was struggling. A confusion matrix provides a detailed breakdown of predictions versus actual labels, making it an invaluable tool for diagnosing model weaknesses in multi-class classification problems.

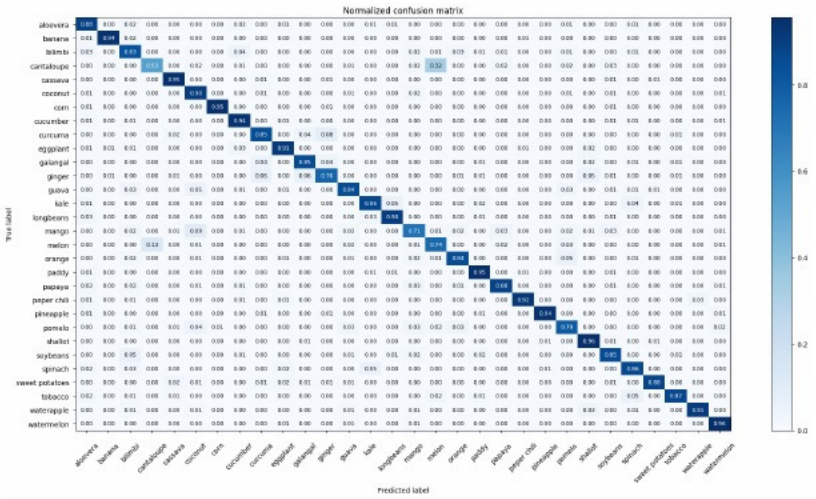

Figure 4: Confusion Matrix

Analysis of the normalised confusion matrix 4 revealed that while most classes achieved high accuracy (above 90%), certain visually similar fruits were frequently confused by the model. The most significant confusion occurred between cantaloupe and melon, where 32% of cantaloupe samples were misclassified as melon, and 13% of melon samples were misclassified as cantaloupe.

This confusion is understandable from a visual perspective, as both fruits share similar characteristics: round shape, similar coloration, and comparable internal flesh appearance. The standard cross-entropy loss function treats all misclassifications equally, providing no mechanism to specifically address such problematic pairs.

3.6 Custom Confusion Penalty Loss

3.6.1 Theoretical Background: Cost-Sensitive Learning

To address specific class confusions, we developed a custom loss function based on **cost-sensitive learning**, a well-established framework formalised by Elkan [2001]. Cost-sensitive learning introduces a cost matrix C where each element C_{ij} represents the penalty for predicting class j when the true class is i. The goal is to minimise the expected cost of misclassification rather than simply minimising the error rate.

The expected cost is defined as:

$$E[\text{Cost}] = \sum_i \sum_j P(i,j) \cdot C_{ij} \tag{1}$$

where $P(i,j)$ is the joint probability of the true class being i and the predicted class being j, and C_{ij} is the associated cost.

In traditional hard cost-sensitive learning, the cost is incurred based on discrete predictions. However, for neural network training, we employ soft cost-sensitive learning, where the cost is weighted by the predicted probability.

3.6.2 Mathematical Formulation

The standard cross-entropy loss for a single sample with true class y is defined as:

$$\mathcal{L}_{CE} = -\log(p_y) \tag{2}$$

where p_y is the predicted probability for the true class y, obtained by applying the softmax function to the network outputs.

Our custom loss function extends cross-entropy by adding a **soft cost-sensitive penalty term**:

$$\mathcal{L}_{total} = \mathcal{L}_{CE} + \mathcal{L}_{penalty} \tag{3}$$

Rather than using a complete $N \times N$ cost matrix, we use a sparse representation that specifies costs only for identified confused pairs. For a set of confused pairs $\mathcal{C} = \{(a, b, C_{ab})\}$, where a and b are class indices and C_{ab} is the associated cost, the penalty term is:

$$\mathcal{L}_{penalty} = \sum_{(a,b,C_{ab})\in\mathcal{C}} C_{ab} \cdot p_b \cdot \mathbb{1}[y = a] + C_{ab} \cdot p_a \cdot \mathbb{1}[y = b] \tag{4}$$

where:

- p_a and p_b are the predicted probabilities for classes a and b respectively
- C_{ab} is the cost (penalty weight) for confusing classes a and b
- $\mathbb{1}[\cdot]$ is the indicator function, equal to 1 if the condition is true and 0 otherwise

This formulation directly implements the soft cost-sensitive learning principle: for a sample with true label $y = a$, the penalty $C_{ab} \cdot p_b$ is proportional to both the cost of the confusion and the probability the model assigns to the confused class. The penalty is applied bidirectionally, meaning both $a \to b$ and $b \to a$ confusions are penalised within a single pair definition.

3.6.3 Experimentation with Penalty Weights

Finding the optimal penalty weight required systematic experimentation. We tested various penalty weight values to observe their effects on the cantaloupe-melon confusion rate and overall model accuracy [1].

Table 1: Single Pair Penalty Results

Penalty	Overall Acc.	Cantaloupe	Melon	Cant. → Mel.	Mel. → Cant.
None	∼87%	56%	74%	30%	13%
3.0	∼87.6%	53%	66%	25%	13%
5.0	90.6%	59%	68%	21%	11%
7.0	87%	47%	54%	17%	10%
12.0	∼86%	41%	45%	5%	3%

The experiments revealed an important trade-off between confusion reduction and class accuracy. A penalty weight of 5.0 emerged as the optimal choice for a single confused pair, achieving the highest overall accuracy (90.6%) while still reducing the cantaloupe-to-melon confusion from 30% to 21%. Higher penalty weights (7.0 and 12.0) further reduced confusion but at the cost of significantly lower class-specific accuracy, as the model became overly cautious about predicting either class.

3.6.4 Per-Pair Weighted Extension

After initial experiments with a single confused pair (cantaloupe-melon), analysis of subsequent confusion matrices revealed a secondary confusion between **melon and pomelo**. This emerged because the model, when discouraged from predicting melon for ambiguous cantaloupe-like images, began predicting pomelo instead—another visually similar round fruit.

To handle multiple confused pairs with different severity levels, we extended the loss function to support per-pair costs. The optimal configuration was determined empirically:

- Cantaloupe ↔ Melon: $C_{ab} = 8.0$
- Melon ↔ Pomelo: $C_{ab} = 5.0$

The higher cost for cantaloupe-melon reflects its status as the primary confusion pair, while the lower cost for melon-pomelo addresses the secondary confusion without over-penalising.

Table 2: Per-Pair Weighted Loss Results

Metric	Baseline	Single Pair (5.0)	Per-Pair (8.0, 5.0)
Overall Accuracy	∼87%	90.6%	∼91.5%
Cantaloupe Accuracy	56%	59%	68%
Melon Accuracy	74%	68%	73%
Cant.→Melon Confusion	30%	21%	16%
Melon→Pomelo Confusion	—	7%	1%

The custom confusion penalty loss function proved effective in reducing specific class confusions while improving overall model accuracy 2. The per-pair weighted variant provided the flexibility needed to address multiple confusions simultaneously with appropriate penalty strengths. This optimised loss function, with weights of 8.0 for cantaloupe-melon and 5.0 for melon-pomelo, served as the foundation for subsequent fine-tuning experiments.

3.7 Unfreezing the model

Despite the improvements from the custom confusion penalty loss, noticeable confusion between cantaloupe and melon remained. To address this, we explored fine-tuning the pretrained ResNet50 layers rather than keeping them entirely frozen.

In transfer learning, early convolutional layers learn general features (edges, textures, shapes) that transfer well across domains, while later layers learn task-specific features Yosinski et al. [2014]. For fine-grained classification tasks, allowing some pretrained layers to adapt can improve performance. We selectively unfroze layer4 while keeping earlier layers frozen, enabling the model to adapt high-level features for fruit classification while preserving general-purpose features from ImageNet.

A critical consideration is the learning rate: using the same rate for pretrained and new layers can destabilise pretrained weights. Following the discriminative fine-tuning approach Howard and Ruder [2018], we implemented differential learning rates—0.01 for the fc layer and a significantly smaller rate for layer4.

The first attempt used a layer4 learning rate of 0.001 (0.1× the fc layer rate) with 5 epochs. The results were disappointing: the model achieved lower accuracy than the frozen version, and new confusions emerged, particularly between cantaloupe/melon and papaya 5.

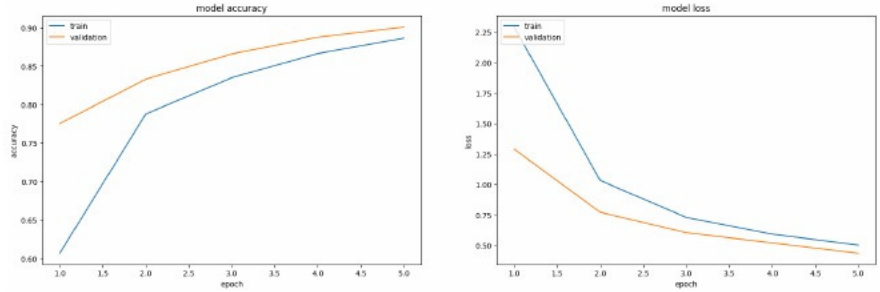

Figure 5: First try of unfreezing the last layer

Analysis of the training curves revealed the issue: the model started at much lower accuracy (60%) compared to the frozen model (89%), indicating that the pretrained features were being disrupted, and the loss started very high. Additionally, 5 epochs was insufficient for the model to recover and converge.

Based on these observations, we adjusted the approach with two key changes: reducing the layer4 learning rate to 0.0001 (0.01× the fc layer rate) for more gentle fine-tuning, and increasing the training duration to 15 epochs to allow sufficient time for convergence.

These modifications yielded better results 6. The training curves showed healthy learning dynamics, with the model steadily improving over the 15 epochs and achieving strong generalisation as evidenced by validation accuracy tracking closely with training accuracy.

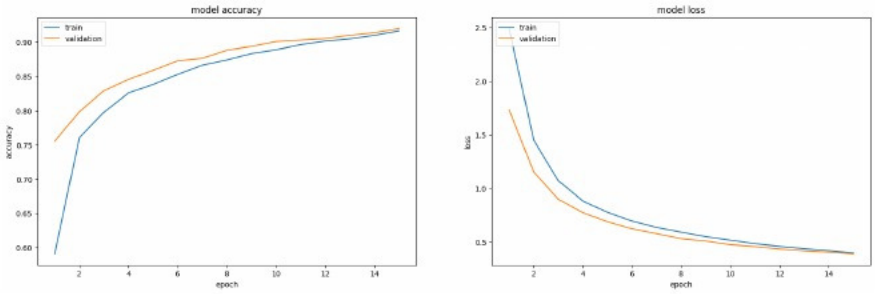

Figure 6: Second try of unfreezing the last layer

Table 3: Effect of Unfreezing Layer4 on Model Performance

Metric	Frozen (5 epochs)	Unfrozen, LR×0.1	Unfrozen, LR×0.01
Epochs	5	5	15
Layer4 Learning Rate	— (frozen)	0.001	0.0001
Overall Accuracy	∼91.5%	∼90%	∼92%
Cantaloupe Accuracy	68%	59%	69%
Melon Accuracy	73%	65%	67%
Cant.→Melon Confusion	16%	11%	8%

3.7.1 Verification: Isolating the Effect of Unfreezing

To confirm that the improvements were due to unfreezing layer4 rather than simply training for more epochs, we conducted a control experiment: training the frozen model for 15 epochs with the same custom loss function.

Table 4: Comparing frozen and unfreezed models with 15 epochs

Metric	Frozen (15 epochs)	Unfrozen (15 epochs)
Overall Accuracy	∼90%	∼92%
Cantaloupe Accuracy	65%	69%
Melon Accuracy	61%	67%
Cant.→Melon Confusion	9%	8%

The comparison confirmed that unfreezing layer4 provides genuine benefits beyond simply training longer 4. With 15 epochs, the unfrozen model achieved 92% accuracy compared to 90% for the frozen model—a 2 percentage point improvement. The unfrozen model also achieved better accuracy for both cantaloupe (69% vs 65%) and melon (67% vs 61%), and lower confusion rates. This result demonstrates that transfer learning strategies can be optimised beyond simple feature extraction when fine-grained classification is required.

4 Results

This section presents the experimental results of our fruit classification model, comparing the baseline model with our improved model that incorporates data augmentation, soft cost-sensitive learning, and partial layer unfreezing.

The **baseline model** was trained using only original images (no data augmentation), standard cross-entropy loss, frozen pretrained layers (only the fully connected layer trainable), and a learning rate of 0.01. The **final model** incorporated all proposed improvements: training with augmented images, soft cost-sensitive loss with per-pair penalty weights, partial unfreezing of layer4, and differential learning rates (0.01 for the fc layer, 0.0001 for layer4).

Final evaluation was performed on a held-out test set that was not used during model development or hyperparameter tuning and we used 15 epochs for both models.

4.1 Overall Performance Comparison

Table 5 summarises the performance comparison between the baseline model and our final optimised model on the test set.

Table 5: Comparison of baseline and final model performance on the test set

Metric	Baseline Model	Final Model
Test Accuracy	80.0%	91.6%
Test Loss	1.0746	0.3819
Correct Predictions	224/280	2490/2718

As expected, test accuracy is slightly lower than validation accuracy for both models, which is normal since the test set contains previously unseen data and had no influence on model development or hyperparameter selection.

The final model achieved a test accuracy of **91.6%**, representing an improvement of **11.6 percentage points** over the baseline model's 80.0% accuracy. Additionally, the test loss decreased by 64% (from 1.0746 to 0.3819), indicating that the final model produces more confident and calibrated predictions.

Notably, the final model shows excellent generalisation: the gap between validation accuracy ($\sim$92%) and test accuracy (91.6%) is minimal, suggesting that the improvements—particularly data augmentation and the regularising effect of soft cost-sensitive learning—helped the model learn robust features that transfer well to unseen data.

5 Discussion

The results of this study successfully validate the proposed iterative workflow for model improvement. Starting from a baseline accuracy of 80.0%, we were able to increase the performance on the held-out test set to 91.6% through a sequence of modifications.

5.1 Interpretation of Findings

The initial gap between training and validation accuracy in the baseline model was effectively closed by the introduction of data augmentation. This confirms findings in that artificially increasing diversity is an effective tool for generalization. A big improvement was the implementation of the cost-sensitive loss function. Standard Cross-Entropy loss penalizes all errors equally, which is suboptimal when dealing with visually ambiguous classes like Cantaloupe and Melon. By introducing a penalty term that specifically targets these pairs, we forced the model to learn more discriminative features for these specific classes. The reduction in Cantaloupe-to-Melon confusion from 30% to 8% demonstrates that loss function engineering can be a powerful alternative to simply adding more data or increasing model size.

Furthermore, the results of the unfreezing experiments highlight the nuances of transfer learning. We observed that high learning rate at the end of these pipelines degrades performance by destroying pretrained weights, while gentle unfreezing of the last convolutional block allows the model to adapt to specific fruit textures without losing general object recognition capabilities.

Despite the improvements, some limitations remain. The model still has 8% confusion rate between Cantaloupe and Melon. Also its worthy of note that the dataset contains images with varied background clutter; while using image augmentation helped, the model may still rely on background context for some predictions rather than the fruit features themselves.

5.2 Future Work

Real-world applications like smart fridges often require identifying multiple fruits in a single image. Extending this classification model to an object detection framework (such as YOLO or Faster R-CNN) would be a logical next step toward practical deployment.

References

Nur-E-Aznin Mimma, Sumon Ahmed, Tahsin Rahman, and Riasat Khan. Fruits classification and detection application using deep learning. *Scientific Programming*, 2022(1), 2022. doi:https://doi.org/10.1155/2022/4194874

Umer Amin, Muhammad Imran Shahzad, Aamir Shahzad, Mohsin Shahzad, Uzair Khan, and Zahid Mahmood. Automatic fruits freshness classification using cnn and transfer learning. *Applied Sciences*, 13(14), 2023. ISSN 2076-3417. doi 10.3390/app13148087. URL https://www.mdpi.com/2076-3417/13/14/8087

Yudha Islam I. Sulistya. Plants-type fruit dataset. Kaggle dataset, 2023. URL https://www.kaggle.com/datasets/yudhaislamisulistya/plants-type-datasets. Accessed: 2025-11-01.

Charles Elkan. The foundations of cost-sensitive learning. In *International Joint Conference on Artificial Intelligence (IJCAI)*, volume 17, pages 973–978. Lawrence Erlbaum Associates Ltd, 2001.

Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pages 3320–3328, 2014.

Jeremy Howard and Sebastian Ruder. Universal language model fine-tuning for text classification. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 328–339, 2018.

A Declaration of Generative AI Usage

I hereby declare that Generative AI tools were used in the preparation of this project. Specifically, the following applications were utilized:

- **Name of AI Application(s):** Gemini (Google), ChatGPT (OpenAI).
- **Purpose of Use:**
  1. **Coding Assistance:** These tools were used as a "dialogue partner" to debug PyTorch code, specifically for troubleshooting dimension mismatch errors in the custom loss function and setting up the differential learning rates.
  2. **Formatting:** AI was used to assist with LaTeX formatting, including generating the code for tables and fixing compilation errors.
  3. **Proofreading:** The text of this report was reviewed by AI to check for grammatical errors and to improve sentence flow.

I confirm that while AI tools were used for support, the experimental design, data analysis, and final conclusions presented in this report are our own work. No content was directly generated by AI without review and modification.